



Teoría de Juegos. Aplicaciones con R

Trabajo Fin de Grado – Grado en Estadística

Autora: Alicia Xiao Casaux Vázquez

Facultad de Matemáticas

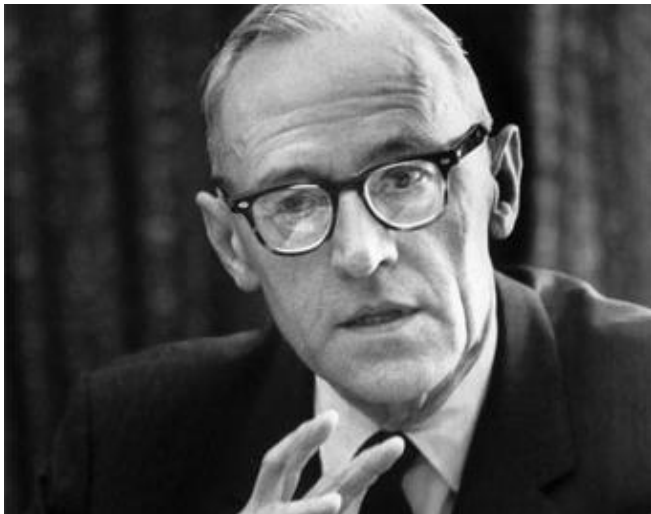
Tutor: José Luis Pino Mejías

Junio 2026

ÍNDICE

- 1 Introducción
- 2 Fundamentos de la Teoría de Juegos
- 3 Concepto de solución
- 4 Implementación en R
- 5 Conclusiones

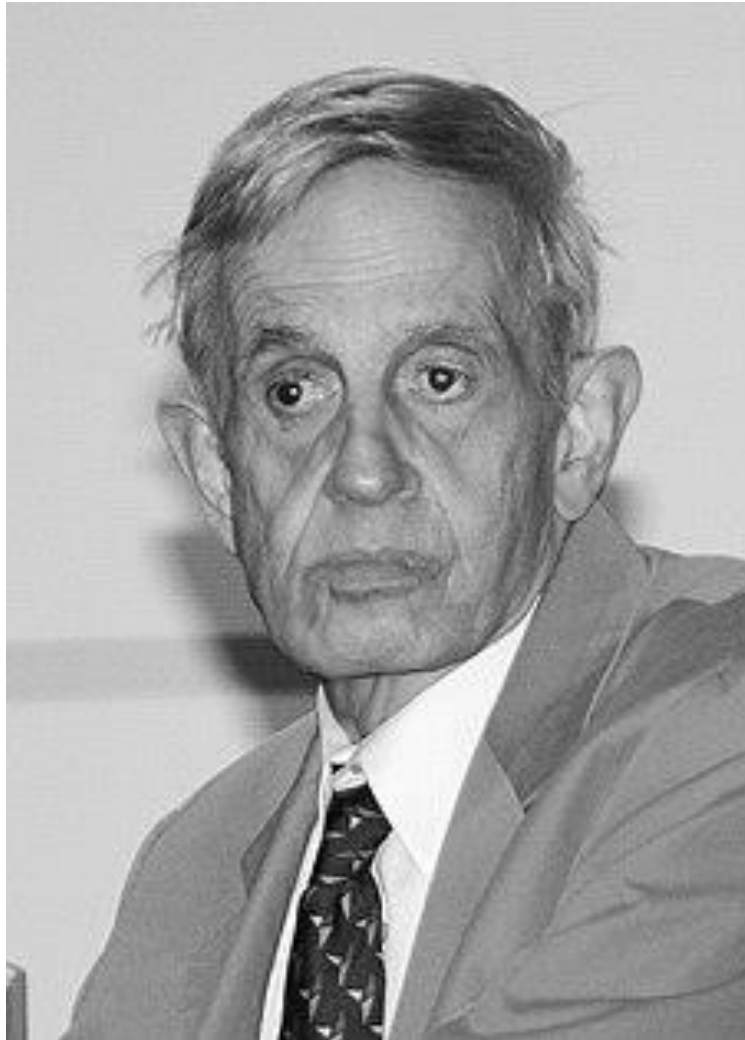
INTRODUCCIÓN



John von Neumann (1903 - 1957)

Oskar Morgenstern (1902 - 1977)

Theory of Games and Economic Behavior (1944)



John Nash (1928 - 2015)

Equilibrio principios 1950

¿Qué es la Teoría de Juegos?

Estudio de situaciones de toma de decisiones estratégicas donde intervienen múltiples agentes y las acciones de uno influyen en los resultados obtenidos de los demás

Aplicaciones

```
graph TD; A[Aplicaciones] --> B[Empresarial]; A --> C[Seguridad]; A --> D[Transporte y gestión de emergencias]; A --> E[Mercados energéticos];
```

Empresarial

Empresas como Apple, Samsung y Amazon emplean EN y estrategias mixtas para analizar la competencia y tomar decisiones estratégicas.

Transporte y gestión de emergencias

Ayuda a optimizar rutas, tráfico y asignación de recursos mediante modelos atacante-defensor.

Seguridad

Sistemas como ARMOR e IRIS utilizan modelos Stackelberg para optimizar controles y asignación de agentes en aeropuertos y vuelos

Mercados energéticos

Para equilibrar oferta y demanda, optimizar precios y planificar infraestructuras como estaciones de carga de vehículos eléctricos.

FUNDAMENTOS

ELEMENTOS FUNDAMENTALES

$$G = (N, S, u)$$



Jugadores (N)

Entidad básica

Individuo o grupo que toma decisiones bajo condiciones de incertidumbre

Buscan maximizar su utilidad



Estrategias (S)

Planificación mental y regla predeterminada

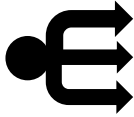
- **Estrategias Puras.** Elección determinista, sin azar
- **Estrategias Mixtas.** La probabilidad influye



Función de Pago (u)

Refleja la utilidad obtenida por cada jugador según el perfil de estrategias elegido

CLASIFICACIÓN DE JUEGOS

 Cooperativo Jugadores forman coaliciones y reparten beneficios	 No Cooperativo Jugadores compiten individualmente
 Suma Nula La ganancia de uno es la pérdida del otro ($u_1 + u_2 = 0$)	 Suma No Nula Beneficios o pérdidas conjuntas son posibles
 Estático Decisiones simultáneas, sin conocer la acción del otro	 Dinámico Decisiones secuenciales en distintos momentos del tiempo
 Información Completa Pagos y estrategias son de dominio público	 Información Incompleta Al menos un jugador desconoce una característica del juego
 Información Perfecta En su turno, el jugador conoce todas las acciones previas	 Información Imperfecta Se desconocen algunas acciones previas

FORMAS DE REPRESENTACIÓN

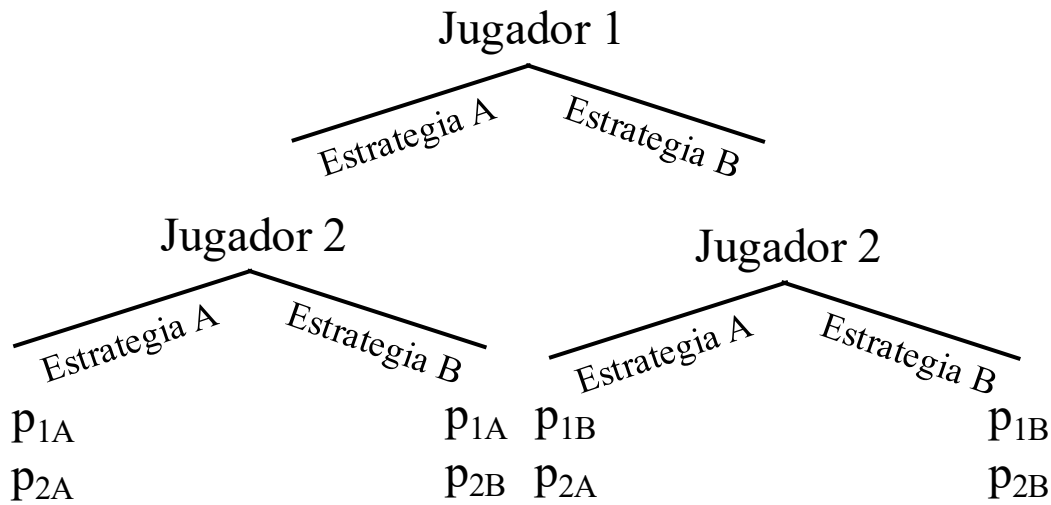
Forma Normal

Utilizada principalmente para juegos estáticos.
Se representa mediante una matriz de pagos donde las filas y columnas son las estrategias de los jugadores.

		Jugador 2	
		Estrategia A	Estrategia B
Jugador 1	Estrategia A	(p_{1A}, p_{2A})	(p_{1B}, p_{2B})
	Estrategia B	(p_{1B}, p_{2A})	(p_{1B}, p_{2B})

Forma Extensiva

Ideal para juegos dinámicos.
Se visualiza mediante un árbol de decisión que permite observar el orden de los movimientos, los nodos de decisión y los pagos finales.



CONCEPTOS DE SOLUCIÓN

ESTRATEGIAS DOMINADAS Y DOMINANTES

Estrategia dominante

Es la mejor respuesta (óptima) independientemente de lo que elijan los demás jugadores.

$$u_i(s_i, s_{-i}) \leq u_i(s'_i, s_{-i})$$

Estrategia estrictamente dominada

Siempre proporciona un resultado inferior a otra alternativa.

$$u_i(s'_i, s_{-i}) < u_i(s''_i, s_{-i})$$

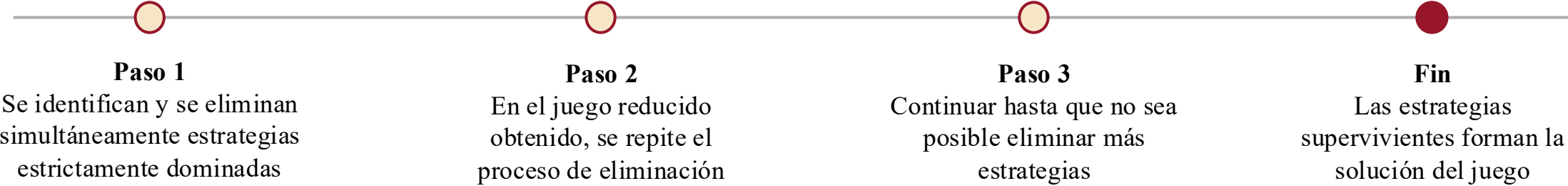
Uso de Estrategias Dominantes (UED)

El UED aplica cuando todos los jugadores poseen una estrategia dominante

		Jugador 2	
		Callar	Confesar
Jugador 1	Callar	(4 , 4)	(0 , 5)
	Confesar	(5 , 0)	(1 , 1)

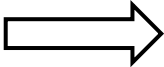
Sol Única UED: (Confesar, Confesar)

ELIMINACIÓN ITERATIVA ESTRICTA



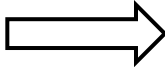
		Jugador 2	
Jugador 1		Izquierda	Derecha
Alta		(4 , 2)	(0 , 1)
Media		(1 , 2)	(2 , 4)
Baja		(3 , 3)	(4 , 2)

Juego G



		Jugador 2
Jugador 1		Izquierda
Alta		(4 , 2)
Baja		(3 , 3)

Juego reducido G₁



		Jugador 2
Jugador 1		Izquierda
Alta		(4 , 2)

Perfil superviviente
S^{EIE}: {Alta, Izquierda}

ELIMINACIÓN ITERATIVA DÉBIL



Paso 1

Se identifican y se eliminan simultáneamente estrategias débilmente dominadas



Paso 2

En el juego reducido obtenido, se repite el proceso de eliminación



Paso 3

Continuar hasta que no sea posible eliminar más estrategias



Fin

Las estrategias supervivientes forman la solución del juego

Jugador 1	Jugador 2		
	I	C	D
A	(3 , 1)	(4 , 2)	(1 , 2)
M	(2 , 4)	(3 , 5)	(4 , 0)
B	(1 , 0)	(2 , 1)	(0 , 3)

Juego G



Jugador 1	Jugador 2	
	C	D
A	(4 , 2)	(1 , 2)
M	(3 , 5)	(4 , 0)

Juego reducido G_1



Jugador 1	Jugador 2
	C
A	(4 , 2)
M	(3 , 5)

Juego reducido G_2



Jugador 1	Jugador 2
	C
A	(4 , 2)

Perfil superviviente $S^{EID}:\{A,C\}$

EQUILIBRIO DE NASH

¿Qué es?

El EN es un perfil de acciones donde ningún jugador tiene incentivos para desviarse unilateralmente.

El perfil de estrategias puras s^* es un EN si

$$u_i(s_i^*, s_{-i}^*) \geq u_i(s_i, s_{-i}^*) \quad \forall s_i \in S_i$$

		Jugador 2	
		Callar	Confesar
Jugador 1	Callar	(4 , 4)	(0 , <u>5</u>)
	Confesar	(<u>5</u> , 0)	(<u>1</u> , <u>1</u>)

Dilema del prisionero

CORRESPONDENCIA DE RESPUESTA ÓPTIMA

Proceso

1. Derivar Óptimos: Calcular la mejor respuesta $R_i(s_{-i})$ de cada jugador ante cualquier combinación.
2. Intersecar: Identificar los EN como los puntos de intersección de todas las correspondencias de respuesta óptima

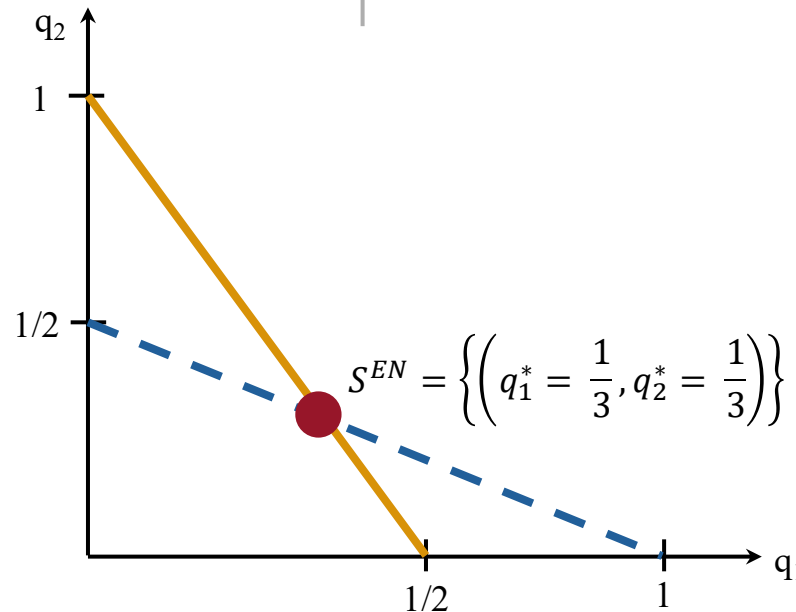
Modelo de Cournot

Maximización de pagos:

$$u_1(q_1, q_2) = q_1(1 - q_1 - q_2)$$

Condición de primer orden ($\partial u_1 / \partial q_1 = 0$):

- Jugador 1: $q_1 = R_1(q_2) = (1 - q_2)/2$
- Jugador 2: $q_2 = R_2(q_1) = (1 - q_1)/2$



ESTRATEGIAS MIXTAS

Una estrategia mixta $\sigma_i \in \Delta(S_i)$ es una distribución de probabilidad sobre el conjunto de estrategias puras S_i ($\sum_j^k \sigma_i^j = 1$)

Se denomina soporte al subconjunto de estrategias puras que reciben una probabilidad estrictamente positiva.

Cálculo en juegos 2x2

		Jugadora 2	
		Cine	Fútbol
Jugador 1	Cine	(<u>1</u> , <u>2</u>)	(0 , 0)
	Fútbol	(0 , 0)	(<u>2</u> , <u>1</u>)

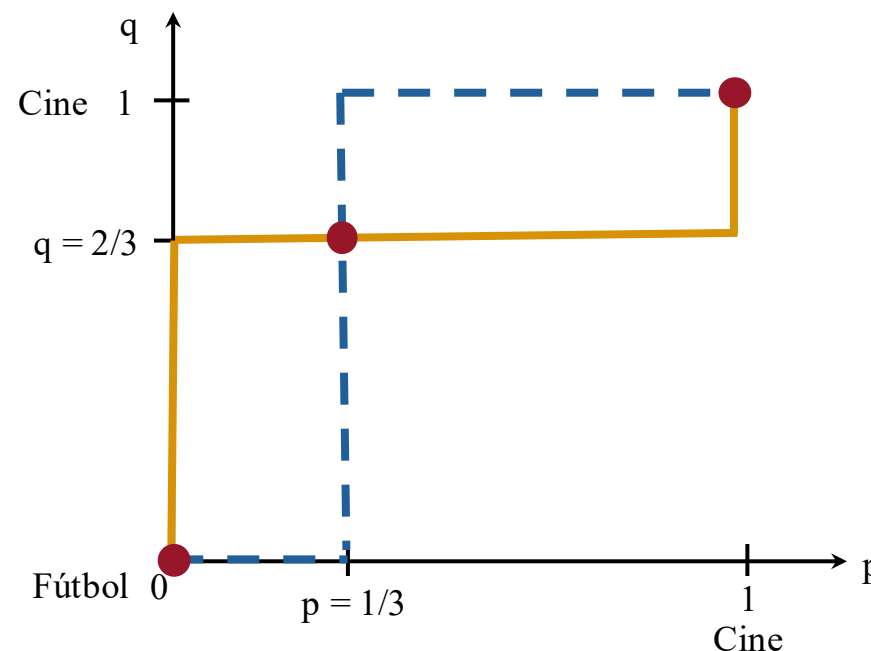
Estrategia mixta para J2: $(q, 1 - q)$

Utilidad J1 (Cine) = $1 \cdot q + 0 \cdot (1 - q) = q$

Utilidad J1 (Fútbol) = $0 \cdot q + 2 \cdot (1 - q) = 2 - 2q$

Igualando se obtiene que $q = 2 - 2q \rightarrow q = 2/3$

Análogamente: $p = 1/3$



Conjunto final

$S^{EN}: \{(Cine, Cine), (Fútbol, Fútbol) \text{ y } [(1/3, 2/3), (2/3, 1/3)]\}$

ÓPTIMO DE PARETO

Un perfil donde es imposible mejorar la situación de un jugador sin empeorar la del otro.

Dilema del prisionero



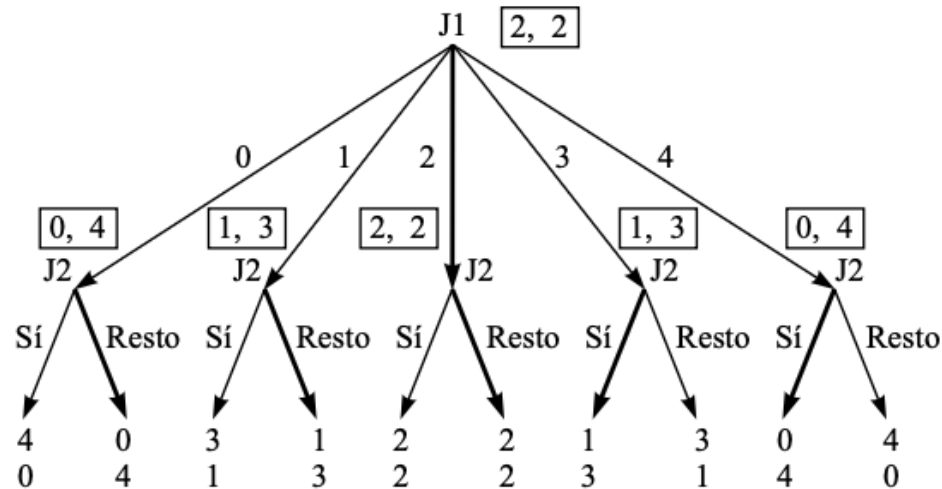
EN PERFECTO EN SUBJUEGOS

Es un refinamiento del equilibrio de Nash para juegos dinámicos con información completa.

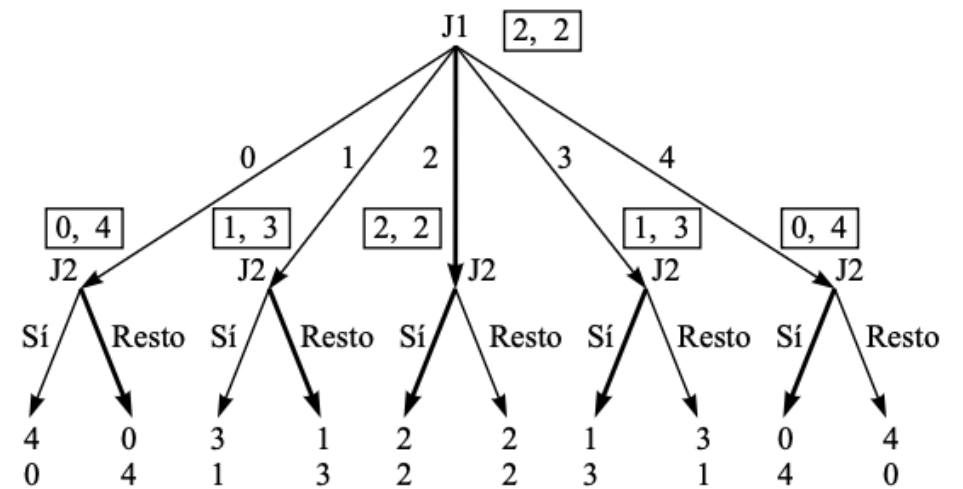
Un ENPS es un perfil de estrategias que es equilibrio de Nash en cada subjuego del juego original

Inducción hacia atrás

(a)



(b)



IMPLEMENTACIÓN EN R

PAQUETES UTILIZADOS

rgamer

Representación de juegos en forma normal y extensiva.

Herramienta visual y didáctica para analizar juegos simples

GNE

Resolución de Equilibrios de Nash generalizados mediante métodos numéricos y condiciones KKT

Gambit/gtree

Interfaz con software externo para la resolución de juegos de gran complejidad secuencial

RGAMER

Forma normal

```
game1 <- normal_form(  
  players = c("Jugador 1", "Jugador 2"),  
  s1 = c("Callar", "Confesar"),  
  s2 = c("Callar", "Confesar"),  
  payoffs1 = c(4, 5, 0, 1),  
  payoffs2 = c(4, 0, 5, 1))
```

Uso de dom() para identificar estrategias dominadas y dominantes.

eliminate_strategy(game, player = "", eliminated = "")

Encontrar EN con solve_nfg(game, mixed = TRUE)

solve_nfg(game1)

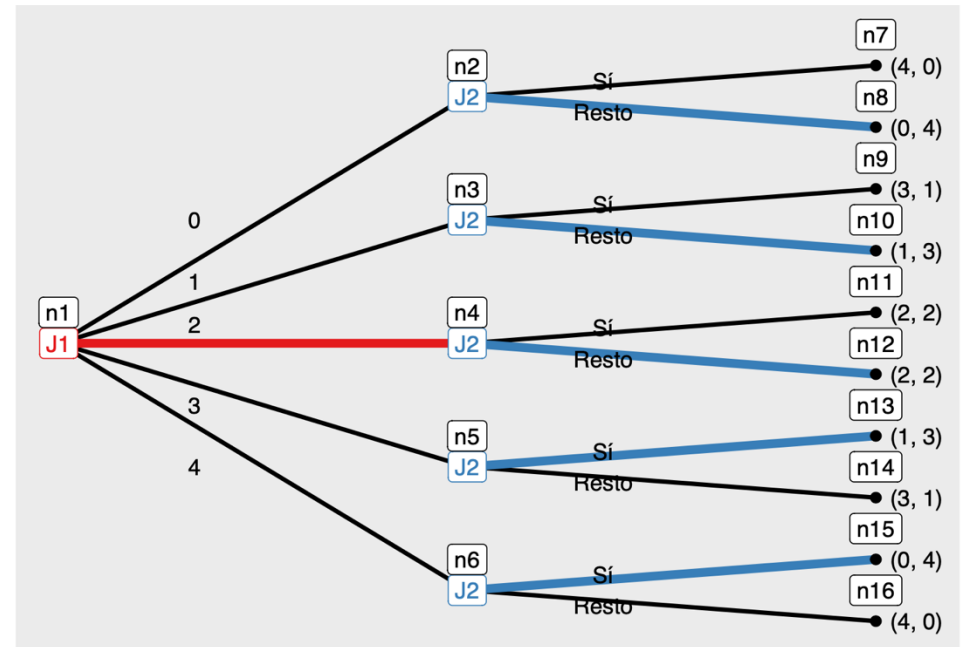
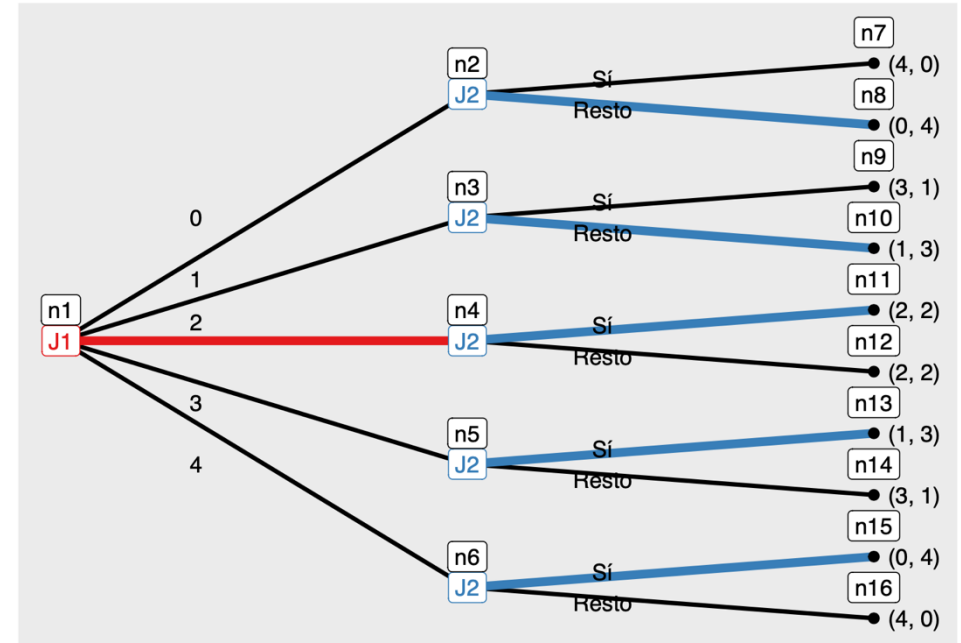
Pure-strategy NE: [Confesar, Confesar]

		Jugador 2	
		Callar	Confesar
Jugador 1	Callar	4, 4	0, 5^
	Confesar	5^, 0	1^, 1^

RGAMER

Forma extensiva

```
game5 <- extensive_form(  
  players = list(  
    "J1",                                # nodo inicial  
    rep("J2", 5),                        # 5 nodos de decisión de J2  
    rep(NA, 10)                          # 10 nodos terminales  
  ),  
  
  actions = list(  
    c("0", "1", "2", "3", "4"),          # acciones de J1  
  
    c("Sí", "Resto"),                   # J2 si J1 juega 0  
    c("Sí", "Resto"),                   # J2 si J1 juega 1  
    c("Sí", "Resto"),                   # J2 si J1 juega 2  
    c("Sí", "Resto"),                   # J2 si J1 juega 3  
    c("Sí", "Resto"),                   # J2 si J1 juega 4  
  
    ),  
  
  payoffs = list(  
    J1 = c(4,0, 3,1, 2,2, 1,3, 0,4),  
    J2 = c(0,4, 1,3, 2,2, 3,1, 4,0)  
  ),  
  direction = "right"  
)  
solve_efg(game5)
```

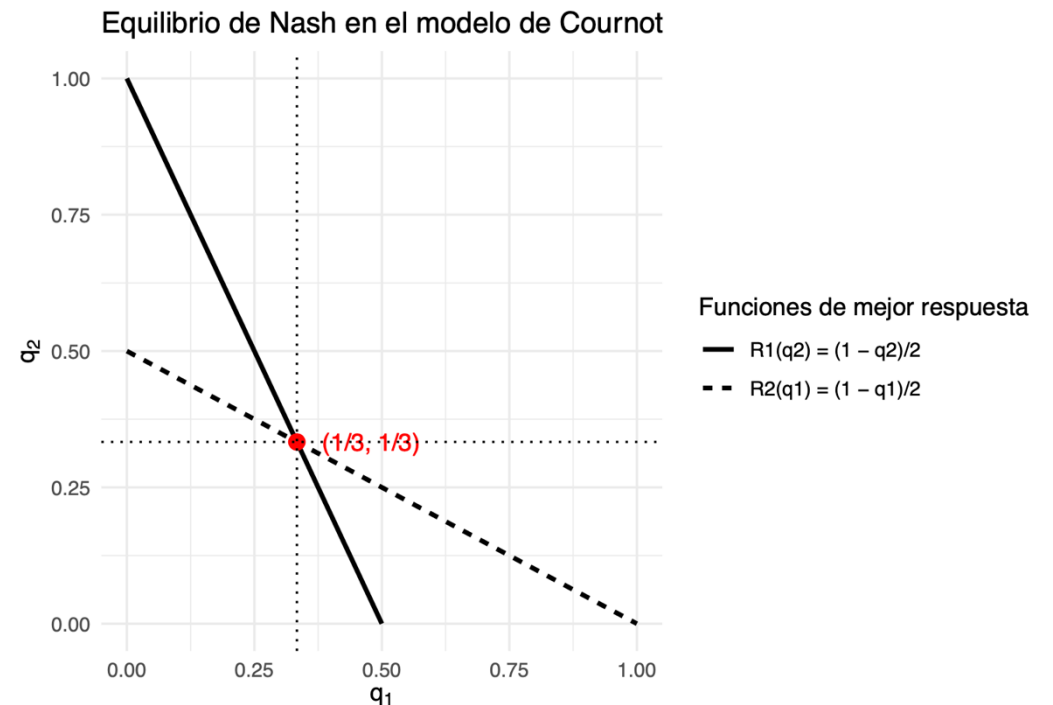


GNE

Función principal GNE(), actúa como una interfaz general delegando en funciones especializadas como GNE.ceq que permite un mayor control sobre la formulación del problema.

```
GNE.ceq(  
  z0, dimx, dimlam,  
  grobj = grobj,  
  heobj = heobj,  
  constr = g,  
  grconstr = grg,  
  heconstr = heg,  
  method = "PR",  
  control = list(trace = 0, maxit = 50)  
)
```

- Método PR (Projection-Residual). Basado en proyecciones.
GNE(after 9 iterations): 0.3333333 0.3333333
- Método AS (Active-Set). Identifica iterativamente restricciones activas
GNE (after 4 iterations): 0.3333333 0.3333333



GAMBIT/GTREE

```
game_gt <- new_game(  
  gameId = "Ultimatum_simple",  
  params = list(numPlayers = 2, cake = 4),  
  
  stages = list(  
  
    stage("oferta",  
      player = 1,  
      actions = list(  
        action("offer", ~0:cake)  
      )  
    ),  
  
    stage("respuesta",  
      player = 2,  
      observe = "offer",  
      actions = list(  
        action("accept", c(FALSE, TRUE))  
      )  
    ),  
  
    stage("pagos",  
      player = 1:2,  
      compute = list(  
        payoff_1 ~ ifelse(accept, cake - offer, 0),  
        payoff_2 ~ ifelse(accept, offer, 0)  
      )  
    )  
  )  
)
```

Se analiza el juego con game_solve()

Con eq_expected_outcomes() se obtienen los resultados

```
## # A tibble: 2 × 7  
##   offer accept payoff_1 payoff_2 util_1 util_2 eqo.ind  
##   <dbl> <dbl>    <dbl>    <dbl> <dbl> <dbl>   <int>  
## 1     1     1        3        1     3     1       1  
## 2     0     1        4        0     4     0       2
```


CONCLUSIONES

¿Preguntas?

Muchas gracias por su atención